

KVM-Based eBPF Rootkit Detection Engine & Toolkit

Design Document

Members:

Luis Abraham - labrahamesco2024@my.fit.edu

Dominick Morales - dmorales2024@my.fit.edu

Braiden Ames - bames2024@my.fit.edu

Dylin Irons - diron2024@my.fit.edu

Advisor:

Dr. Ribeiro - eribeiro@fit.edu

Client:

Amy Alvarez - NASA Scientist

Florida Institute of Technology Senior Design

Table of Contents

- 1. Introduction**
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Definitions, Acronyms, and Abbreviations
 - 1.4. References
- 2. pKVM Design**
 - 2.1. Architecture Overview
 - 2.2. Userspace and EL1 Interface
 - 2.3. EL2 pKVM Module
 - 2.4. Communication and Security
- 3. RootView Design**
 - 3.1. Architecture Overview
 - 3.2. Low-Level VMI Abstraction
 - 3.3. Linux and eBPF Introspection
 - 3.4. VM Exit Monitoring
 - 3.5. Python Toolkit and Detection Engine
 - 3.6. VM Management
- 4. Web UI Design**
 - 4.1. Architecture Overview
 - 4.2. User Interface
 - 4.3. Monitoring and Detection Visualization
 - 4.4. Backend Communication
- 5. System Integration**
 - 5.1. Cross-Architecture Integration
 - 5.2. Data and Event Flow

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements of RootView, a virtual machine introspection platform designed to support the observation and eventual detection of malicious activity within the Linux kernel, with an emphasis on eBPF-based malware and rootkits.

The document establishes the expected capabilities, interfaces, constraints, and performance characteristics of the RootView system. It will serve as a reference for implementation, testing, and evaluation throughout the senior design project.

1.2 Scope

RootView is a research platform that uses virtualization-based introspection to observe Linux guest systems from outside the guest operating system. The platform aims to provide researchers with tools for examining guest memory, processor state, kernel structures, processes, and eBPF activity without relying exclusively on software executing within the monitored guest.

The system will support multiple virtualization architectures. On x86-64 systems, RootView will utilize KVM, QEMU, and KVMi to access and inspect guest state. On ARM64 systems, RootView will investigate and implement pKVM extensions that enable virtual machine introspection through the EL2 hypervisor environment.

RootView will also incorporate VM exit hooks to observe selected events occurring during guest execution. These observations will supplement traditional VMI capabilities and provide additional information about guest behavior.

The initial system will focus on establishing reliable VM management, introspection, and observation capabilities. Future development will expand these capabilities into a reusable research API and an eBPF malware detection engine that correlates hypervisor-level observations with guest kernel telemetry.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
VMI	Virtual Machine Introspection
KVM	Kernel-based Virtual Machine
KVMi	KVM Virtual Machine Introspection interface
pKVM	Protected KVM
EL2	ARM Exception Level 2, associated with hypervisor execution

eBPF	Extended Berkeley Packet Filter
BTF	BPF Type Format
DWARF	Debugging information format used to describe program and data structures
VM exit	A transition from guest execution to hypervisor execution
VM exit hook	A mechanism for observing or handling selected VM exit events
Guest	The virtual machine being monitored
Hypervisor	The virtualization layer responsible for managing guest execution
Kernel profile	Information used to interpret Linux kernel structures and symbols

1.4 References

- KVM documentation and implementation.
- LibVMI documentation.
- Linux KVM documentation.
- Linux pKVM documentation and relevant ARM virtualization documentation.
- Linux eBPF and BTF documentation.
- RootView project plan and design document.

2. pKVM Design

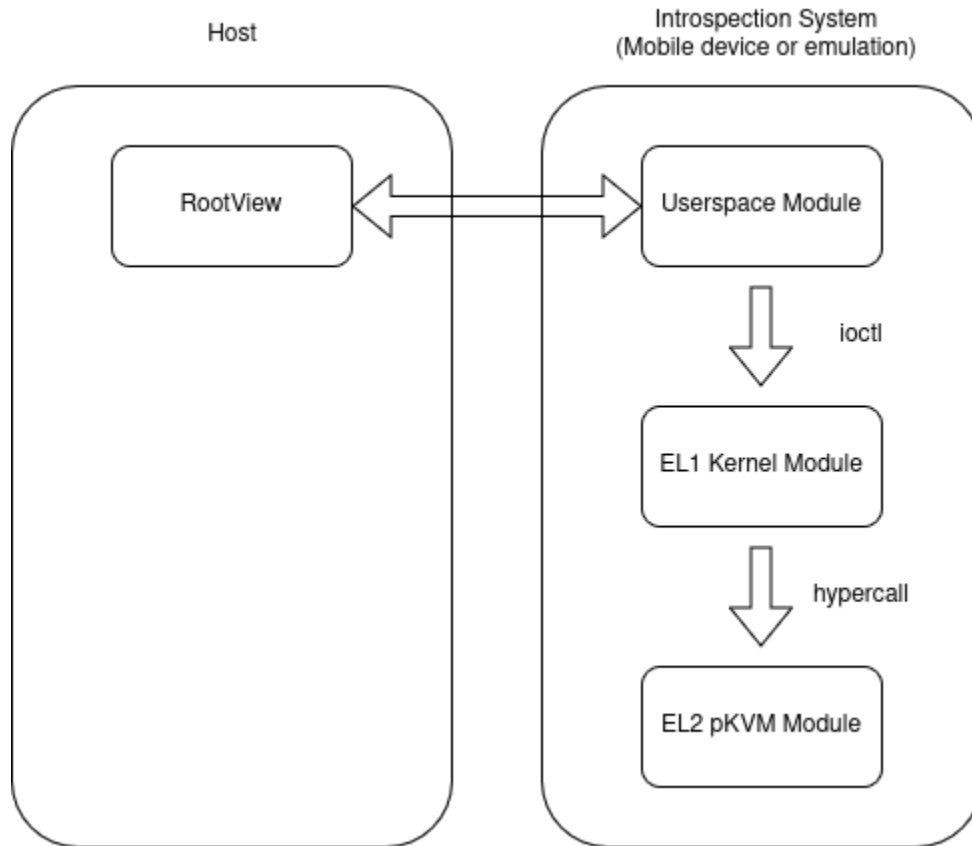
2.1 Architecture Overview

pKVM is an ARM64 virtual machine introspection backend being developed for RootView. It will extend the capabilities of the project to support introspection on ARM-based systems using Protected KVM (pKVM), specifically mobile devices. Unlike the x86-64 implementation, which utilizes KVM, pKVM will require custom components that communicate across userspace, the EL1 kernel, and the EL2 hypervisor.

The architecture consists of a userspace module, an EL1 kernel module, and an EL2 pKVM module. These components will work together to provide RootView with access to guest memory and processor state while maintaining the isolation properties of the pKVM environment.

Because no existing introspection tooling equivalent to KVM currently exists for pKVM, the EL2 module was implemented from scratch. Development is validated first

against an emulated EL2 environment, then confirmed on physical hardware — specifically an unlocked Google Pixel 7, whose manufacturer-provided ability to load custom kernel modules (normally intended for hardware bring-up) is repurposed here to obtain research introspection access.



2.2 Userspace and EL1 Interface

The userspace module will provide the interface between RootView and the pKVM implementation. It will receive introspection requests from RootView and communicate with the EL1 kernel module through IOCTL operations.

The EL1 kernel module will serve as an intermediary between userspace and the EL2 hypervisor. It will process incoming requests and invoke the appropriate hypercalls to request introspection operations from the EL2 module. Responses will be returned through the same communication path to the userspace module and ultimately to RootView.

2.3 EL2 pKVM Module

The EL2 pKVM module will be responsible for performing the low-level introspection operations required by RootView. It will handle requests received through hypercalls and provide access to supported guest state.

Initial capabilities will include guest memory access and retrieval of selected processor state. Additional operations may be implemented as the project develops. The module will be designed to provide the necessary introspection functionality while preserving the security and isolation guarantees of pKVM.

2.4 Communication and Security

Communication between the pKVM components will follow a request-and-response model. RootView will issue requests through the userspace module, which will forward them to EL1. The EL1 module will then communicate with EL2 through hypercalls and return the resulting information to RootView.

The implementation will account for privilege boundaries between userspace, EL1, and EL2. Requests will be validated before being passed to the hypervisor, and access to guest state will be restricted to supported introspection operations.

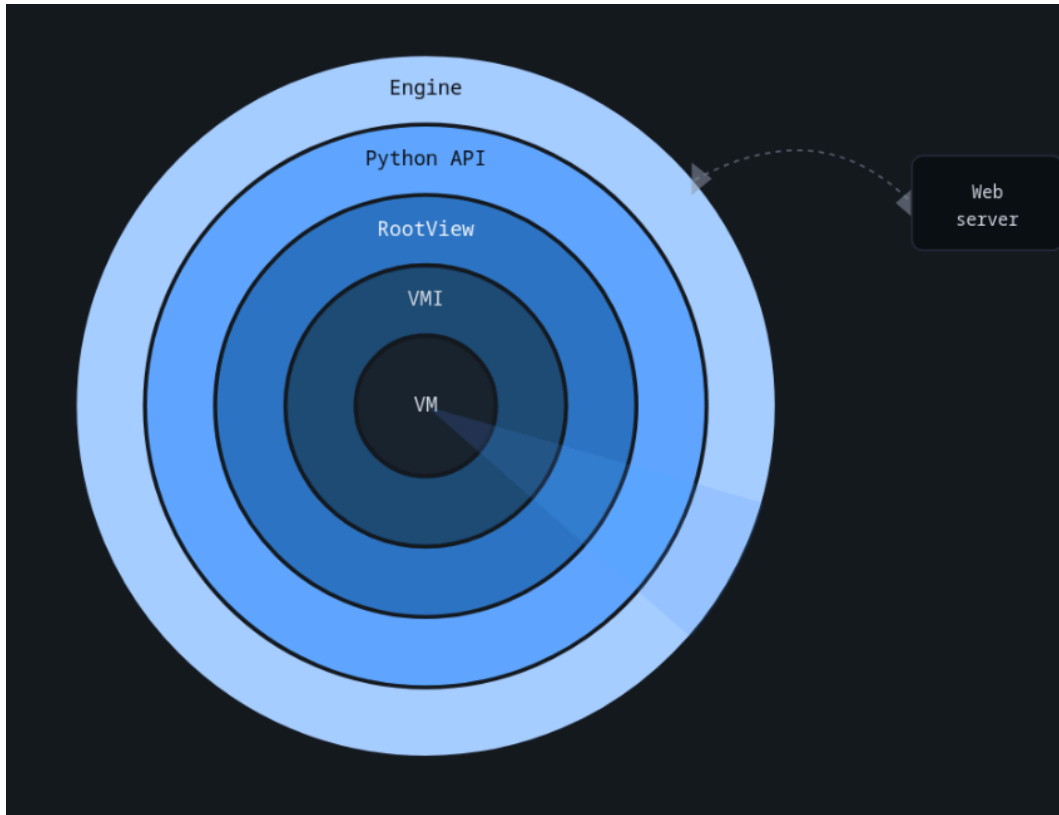
This approach deliberately grants userspace-directed introspection access into EL2, which circumvents the isolation guarantees pKVM is designed to enforce. It is therefore limited to research use on unlocked, non-production devices and must not be deployed on locked-bootloader or production hardware.

3. RootView Design

3.1 Architecture Overview

RootView is the primary research toolkit and detection engine built on top of the underlying introspection infrastructure. Its architecture is divided into several layers, each responsible for a specific aspect of virtual machine introspection and analysis.

The low-level VMI abstraction will provide a common interface for interacting with KVM and pKVM. Above this layer, Linux and eBPF introspection components will interpret guest state and provide higher-level information. Python bindings will expose these capabilities as a reusable research toolkit, which will serve as the foundation for the eBPF malware detection engine.



3.2 Low-Level VMI Abstraction

The low-level VMI abstraction will provide a common C/C++ interface for interacting with the supported introspection backends. This layer will abstract backend-specific implementation details and allow higher-level components to perform introspection without directly interacting with KVM or pKVM.

The interface will provide operations for reading and writing guest memory, retrieving processor state, and performing other supported low-level introspection tasks. Backend-specific differences will be handled internally, allowing the remainder of RootView to use a consistent interface across supported architectures.

3.3 Linux and eBPF Introspection

The Linux introspection layer will interpret raw guest memory and processor state to provide meaningful information about the guest operating system. It will utilize kernel profiles, BTF/DWARF information, and kernel symbols to identify and interpret Linux kernel structures.

This layer will support process enumeration, kernel object inspection, and address translation. It will also provide the foundation for eBPF introspection by identifying and examining BPF programs, maps, and attachments within the guest.

The eBPF component will collect relevant telemetry and represent observations in a standardized format. These observations will be made available to higher-level components for analysis and correlation with external VMI data.

3.4 VM Exit Monitoring

RootView will incorporate VM exit hooks to observe selected events occurring during guest execution. These hooks have already been implemented and will serve as an additional source of information for the introspection platform.

The monitoring component will capture relevant information from supported VM exit events and make it available to the RootView backend. This information may be used alongside memory introspection and eBPF telemetry to provide a more comprehensive view of guest activity.

3.5 Python Toolkit and Detection Engine

Python bindings will expose the functionality of the C/C++ backend through a high-level research interface. The toolkit will allow users to manage virtual machines and perform supported memory, Linux, and eBPF introspection operations without directly interacting with the underlying virtualization interfaces.

The detection engine will be developed using this toolkit. It will consume introspection results and collected telemetry to identify suspicious activity within the guest. By correlating observations from different sources, the engine will support the development and evaluation of eBPF malware detection techniques.

3.6 VM Management

RootView will provide tools for configuring and managing virtual machines used for introspection. These tools will support VM setup, configuration, and lifecycle management through QEMU and KVM.

The VM management component will also coordinate the initialization of the appropriate introspection backend based on the supported virtualization environment.

4. Web UI Design

4.1 Architecture Overview

The RootView web interface will provide a user-facing method for interacting with the introspection toolkit and detection engine. It will consist of a frontend and a backend service that communicates with the RootView Python toolkit.

The backend service will handle requests from the frontend and invoke the appropriate RootView functionality. This separation will allow the web interface to interact with privileged host-side operations without directly accessing the underlying hypervisor interfaces.

4.2 User Interface

The web interface will provide a dashboard for managing virtual machines and accessing introspection functionality. Users will be able to select monitored VMs, view their status, and request supported introspection operations.

The interface will present Linux kernel and eBPF information in a structured format, allowing users to examine processes, kernel objects, BPF programs, maps, and attachments without manually interpreting raw memory.

4.3 Monitoring and Detection Visualization

The web interface will provide views for displaying VM exit events, eBPF telemetry, and detection results. Researchers will be able to examine collected observations and access relevant information associated with suspicious activity.

The interface will aim to make RootView's introspection and detection capabilities accessible to researchers through an organized and interactive environment.

4.4 Backend Communication

The web frontend will communicate with the backend service through a defined application interface. The backend will translate user requests into operations supported by the Python toolkit and return the resulting information to the frontend.

The communication design will account for error handling and potentially long-running introspection operations. Additional mechanisms for live event updates may be incorporated as the monitoring functionality develops.

5. System Integration

5.1 Cross-Architecture Integration

RootView will integrate KVM and pKVM through the low-level VMI abstraction layer. This design will allow higher-level components to access common introspection functionality without depending on a specific virtualization architecture.

Architecture-specific differences, including memory translation and processor state representation, will be handled by the respective backend implementations.

5.2 Data and Event Flow

Introspection requests will originate from the toolkit or user interface and pass through the RootView backend to the appropriate VMI implementation. The selected backend will perform the requested operation and return the result to the calling component.

VM exit events and eBPF telemetry will be collected independently and made available to the detection engine. The engine will correlate these observations to identify suspicious activity and produce results that can be accessed through the toolkit and web interface.

